Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е. А.

Проверил:

Никитин С. К.

Брест 2026

Цель работы: Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Ход работы:

1. Сгенерировать ряд по формуле варианта и построить его график с отметкой границы между обучающим и тестовым участками.
2. Нормализовать ряд по минимуму и максимуму **обучающего участка**: x_norm = (x - x_min) / (x_max - x_min). Записать формулу обратного пересчёта и использовать её при вычислении всех метрик в исходной шкале. **Объяснить**, почему min/max нельзя брать по всему ряду.
3. Сформировать обучающую и тестовую выборки методом скользящего окна размера p (вход – p значений, цель – следующее значение). Указать количество получившихся примеров в каждой выборке.
4. Реализовать многослойный персептрон p–N–1:
    - скрытый слой – сигмоида;
    - выходной слой – линейный (задача регрессии), f(s) = s;
    - функция потерь – MSE;
    - обучение – онлайн-режим, перемешивание порядка примеров в каждой эпохе;
    - вывести формулы градиентов для линейного выходного слоя и сигмоидного скрытого слоя и привести их в отчёте.

5. Критерий остановки: средняя по обучающей выборке ошибка $E_s \le E_e$, где $E_e = 10^{-4}$ (нормализованная шкала), либо достижение 20 000 эпох. Скорость обучения подобрать самостоятельно из диапазона 0.01–0.3 и обосновать выбор.
6. **Базовый эксперимент.** Обучить сеть с параметрами p и N из своего варианта. Зафиксировать число эпох, итоговую Es, а также метрики на обучающем и тестовом участках **в исходной шкале**: MAE, RMSE и MAPE (если ряд проходит через ноль – вместо MAPE использовать sMAPE, и пояснить почему).
7. **Исследование окна.** При фиксированном N из варианта обучить сеть для $p \in \{2, 4, p_{вар}, 2 \cdot p_{вар}\}$. Для каждого p выполнить по 5 запусков с разными seed и свести в таблицу: среднее и разброс числа эпох, среднее и разброс тестового MAE, число запусков, не достигших критерия остановки.
8. **Исследование скрытого слоя.** При фиксированном p из варианта обучить сеть для $N \in \{1, 2, N_{вар}, 2 \cdot N_{вар}\}$. Выполнить ту же серию из 5 запусков и заполнить аналогичную таблицу.
9. **Многошаговый прогноз.** Для лучшей по тестовому MAE конфигурации выполнить рекурсивный прогноз на 30 шагов вперёд, начиная с k = 140: каждое предсказание подставляется в окно как входное значение для следующего шага. Построить график «эталон – рекурсивный прогноз» и график абсолютной ошибки в зависимости от номера шага. Определить, на каком шаге ошибка превышает 5 % от размаха ряда.
10. Построить графики:
    - ряд с границей train/test и одношаговым прогнозом сети на обоих участках на одних осях;
    - сходимость (Es от эпохи) для базовой конфигурации и для двух конфигураций из исследования окна на одних осях (логарифмическая шкала по Y);
    - диаграмма рассеяния «эталон – прогноз» на тестовом участке с диагональю y = x;
    - рекурсивный прогноз и рост ошибки по шагам (п. 9).

11. Реализовать режим функционирования: пользователь вводит p последовательных значений ряда в исходной шкале (либо номер k, с которого брать окно из ряда). Программа выводит:

- нормализованные входы;
- прогноз в нормализованной шкале (4 знака после запятой);
- прогноз в исходной шкале;
- истинное значение x(k) и абсолютную ошибку (если окно взято из ряда).

Продемонстрировать работу не менее чем на 3 окнах из обучающего участка, 3 окнах из тестового участка и 1–2 пользовательских окнах (в том числе с «выбросом»).

12. Вставить в отчет фото/скан рукописных расчётов. Вверху листа от руки должны быть написаны Ваша Фамилия, Инициалы, номер группы, вариант и дата выполнения:

- Графическая схема пайплайна (Блок-схема). Составьте корректную цепочку обработки данных, используя предложенный набор блоков (приложение 1). Объясните свой выбор.
- Ручной расчёт 1 шага обучения. На отдельном листе бумаги от руки распишите один полный шаг (Forward + Backward pass) для сети 3–2–1 со следующими параметрами, привязанными к вашему варианту V:
  - Входы: X = [0.2, 0.5, 0.8], Цель: d = 0.6
  - Веса скрытого слоя: W11 = 0.05*V, W12 = -0.2, W21 = 0.3, W22 = 0.05*V, W31 = -0.1, W32 = 0.4 (b1=0.1, b2=-0.1)
  - Веса выходного слоя: W_out1 = 0.5, W_out2 = -0.2*V (b_out = 0.2)

Вариант	a	b	c	d	p	N
8	1.8	0.45	0.9	1.00	6	5

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

A, B, C, D = 1.8, 0.45, 0.9, 1.0
P, N = 6, 5
K_TOTAL = 200
K_TRAIN = 140
Lr = 0.1
Ee = 1e-4
MAX_EPOCHS = 20000
SEED = 42

def generate_series(a, b, c, d, n = K_TOTAL):
    k = np.arange(n)
    return a * np.cos(b * k) + c * np.sin(d * k)

series = generate_series(A, B, C, D)

plt.figure(figsize=(11, 4))
plt.plot(series, label="x(k)")
plt.axvline(K_TRAIN - 0.5, color="r", linestyle="--", label="граница train/test")
plt.xlabel("k"); plt.ylabel("x(k)"); plt.title("Ряд x(k)")
plt.legend(); plt.grid(alpha=0.3)
plt.show()

def fit_minmax(x_train):
```

```
    return x_train.min(), x_train.max()

def normalize(x, x_min, x_max):
    return (x - x_min) / (x_max - x_min)

def denormalize(xn, x_min, x_max):
    return xn * (x_max - x_min) + x_min

x_min, x_max = fit_minmax(series[:K_TRAIN])
series_n = normalize(series, x_min, x_max)
print(f"x_min = {x_min:.4f}, x_max = {x_max:.4f}")
print(f"train в [{series_n[:K_TRAIN].min():.3f}; {series_n[:K_TRAIN].max():.3f}]")
print(f"test в [{series_n[K_TRAIN:].min():.3f}; {series_n[K_TRAIN:].max():.3f}]")

def make_windows(x, p, t_start, t_end):
    X = np.array([x[t - p:t] for t in range(t_start, t_end)])
    y = np.array([x[t] for t in range(t_start, t_end)])
    return X, y

Xtr, ytr = make_windows(series_n, P, P, K_TRAIN)
Xte, yte = make_windows(series_n, P, K_TRAIN, K_TOTAL)
print("train:", Xtr.shape, ytr.shape, "-> примеров:", len(Xtr))
print("test:", Xte.shape, yte.shape, "-> примеров:", len(Xte))

def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-s))

def init_weights(p, n, rng, scale=0.3):
    return {"W1": rng.uniform(-scale, scale, (n, p)),
            "b1": rng.uniform(-scale, scale, n),
            "W2": rng.uniform(-scale, scale, n),
            "b2": float(rng.uniform(-scale, scale))}

def forward(w, x):
    h = sigmoid(x @ w["W1"].T + w["b1"])
    y = h @ w["W2"] + w["b2"]
    return h, y

def backward(w, x, h, y, d):
    delta_out = 2.0 * (y - d)
    dW2 = delta_out * h
    db2 = delta_out
    delta_h = delta_out * w["W2"] * h * (1.0 - h)
    dW1 = np.outer(delta_h, x)
    db1 = delta_h
    return dW1, db1, dW2, db2

def mse(w, X, d):
    _, y = forward(w, X)
    return float(np.mean((y - d) ** 2))

def train(Xtr, ytr, p, n, seed, lr = Lr, ee = Ee, max_epochs = MAX_EPOCHS):
    rng = np.random.default_rng(seed)
    w = init_weights(p, n, rng)
    history = []
    for epoch in range(1, max_epochs + 1):
        for i in rng.permutation(len(Xtr)):
            x, d = Xtr[i], ytr[i]
            h, y = forward(w, x)
            dW1, db1, dW2, db2 = backward(w, x, h, y, d)
```

```
            w["W1"] -= lr * dW1
            w["b1"] -= lr * db1
            w["W2"] -= lr * dW2
            w["b2"] -= lr * db2
        Es = mse(w, Xtr, ytr)
        history.append(Es)
        if Es <= ee:
            return w, epoch, history, True
    return w, max_epochs, history, False

def mae(y, yh):   return float(np.mean(np.abs(y - yh)))
def rmse(y, yh):  return float(np.sqrt(np.mean((y - yh) ** 2)))
def mape(y, yh):  return float(100 * np.mean(np.abs((y - yh) / y)))
def smape(y, yh): return float(100 * np.mean(2 * np.abs(y - yh) / (np.abs(y) + np.abs(yh))))

def predict_orig(w, Xn, x_min, x_max):
    _, yn = forward(w, Xn)
    return denormalize(yn, x_min, x_max)

def evaluate(w, Xn, y_true, x_min, x_max):
    yh = predict_orig(w, Xn, x_min, x_max)
    return {"MAE": mae(y_true, yh), "RMSE": rmse(y_true, yh),
            "sMAPE": smape(y_true, yh), "MAPE": mape(y_true, yh)}

print(" lr   | эпохи (ср ± σ) | test MAE | не сошлись")
for lr in [0.01, 0.03, 0.05, 0.1, 0.2, 0.3]:
    eps, maes, fails = [], [], 0
    for s in [1, 2, 3, 4, 5]:
        w, ep, hist, ok = train(Xtr, ytr, P, N, s, lr = lr)
        eps.append(ep)
        maes.append(evaluate(w, Xte, series[K_TRAIN:], x_min, x_max)["MAE"])
        fails += (not ok)
    print(f"{lr:5.2f} | {np.mean(eps):6.1f} ± {np.std(eps):5.1f} | {np.mean(maes):.4f}   |
{fails}")

w_base, ep_base, hist_base, ok_base = train(Xtr, ytr, P, N, SEED)
print(f"Эпох: {ep_base}, Es = {hist_base[-1]:.3e}, критерий достигнут: {ok_base}")
for name, X, y_true in [("train", Xtr, series[P:K_TRAIN]),
                        ("test ", Xte, series[K_TRAIN:])]:
    m = evaluate(w_base, X, y_true, x_min, x_max)
    print(f"{name}: MAE={m['MAE']:.4f}  RMSE={m['RMSE']:.4f}  "
          f"sMAPE={m['sMAPE']:.2f}%  (MAPE={m['MAPE']:.1f}% — раздувается из-за нулей)")

SEEDS = [1, 2, 3, 4, 5]

def run_config(p, n, seeds=SEEDS):
    Xtr_, ytr_ = make_windows(series_n, p, p, K_TRAIN)
    Xte_, _ = make_windows(series_n, p, K_TRAIN, K_TOTAL)
    runs = []
    for s in seeds:
        w, ep, hist, ok = train(Xtr_, ytr_, p, n, s)
        m = evaluate(w, Xte_, series[K_TRAIN:], x_min, x_max)
        runs.append({"p": p, "n": n, "seed": s, "w": w, "epochs": ep,
                     "hist": hist, "ok": ok, "test_mae": m["MAE"]})
    return runs

def print_table(title, name, table):
    print(title)
    print(f"{name:>4} | эпохи: ср. ± σ  | test MAE: ср. ± σ | не достигли Es≤Ee (из 5)")
    for key, runs in table.items():
```

```
        ep = np.array([r["epochs"] for r in runs])
        ma = np.array([r["test_mae"] for r in runs])
        fails = sum(not r["ok"] for r in runs)
        print(f"{key:>4} | {ep.mean():8.1f} ± {ep.std():6.1f} | {ma.mean():.4f} ±
{ma.std():.4f}  | {fails}")
    print()

window_runs = {p: run_config(p, N) for p in [2, 4, P, 2 * P]}
neuron_runs = {n: (window_runs[P] if n == N else run_config(P, n))
               for n in [1, 2, N, 2 * N]}

print_table(f"Исследование окна (N = {N})", "p", window_runs)
print_table(f"Исследование скрытого слоя (p = {P})", "N", neuron_runs)

configs = {}
for p_, runs in window_runs.items():
    configs[(p_, N)] = runs
for n_, runs in neuron_runs.items():
    configs[(P, n_)] = runs
best_key = min(configs, key=lambda k: np.mean([r["test_mae"] for r in configs[k]]))
best_run = min(configs[best_key], key=lambda r: r["test_mae"])
print(f"Лучшая конфигурация по среднему test MAE: p = {best_key[0]}, N = {best_key[1]}")
print(f"Берём её лучший запуск: seed = {best_run['seed']}, test MAE =
{best_run['test_mae']:.4f}")

def recursive_forecast(w, series_n, p, start, steps):
    window = list(series_n[start - p:start])
    out = []
    for _ in range(steps):
        _, y = forward(w, np.array(window))
        out.append(float(y))
        window = window[1:] + [float(y)]
    return np.array(out)

STEPS = 30
p_best = best_run["p"]
rec_pred = denormalize(recursive_forecast(best_run["w"], series_n, p_best, K_TRAIN, STEPS),
x_min, x_max)
rec_true = series[K_TRAIN:K_TRAIN + STEPS]
rec_err = np.abs(rec_true - rec_pred)
threshold = 0.05 * (series.max() - series.min())
bad = np.where(rec_err > threshold)[0]
print(f"Порог 5% размаха: {threshold:.4f}")
if len(bad):
    print(f"Ошибка впервые превышает порог на шаге {bad[0] + 1} (k = {K_TRAIN + bad[0]})")
else:
    print("За 30 шагов ошибка порог не превысила")

Xtr, ytr = make_windows(series_n, P, P, K_TRAIN)
Xte, yte = make_windows(series_n, P, K_TRAIN, K_TOTAL)
pred_tr = predict_orig(w_base, Xtr, x_min, x_max)
pred_te = predict_orig(w_base, Xte, x_min, x_max)

plt.figure(figsize=(12, 4))
plt.plot(series, "k", label="эталон")
plt.plot(range(P, K_TRAIN), pred_tr, "b--", label="прогноз (train)")
plt.plot(range(K_TRAIN, K_TOTAL), pred_te, "r--", label="прогноз (test)")
plt.axvline(K_TRAIN - 0.5, color="gray", linestyle=":")
plt.xlabel("k"); plt.ylabel("x(k)"); plt.title("Одношаговый прогноз, базовая сеть")
plt.legend(); plt.grid(alpha=0.3); plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.semilogy(hist_base, label=f"базовая: p={P}, N={N}")
for p_ in (2, 2 * P):
    plt.semilogy(window_runs[p_][0]["hist"], label=f"p={p_}, N={N} (seed={SEEDS[0]})")
plt.axhline(Ee, color="r", linestyle=":", label="Ee = 1e-4")
plt.xlabel("эпоха"); plt.ylabel("Es (лог. шкала)"); plt.title("Сходимость")
plt.legend(); plt.grid(alpha=0.3, which="both"); plt.show()

plt.figure(figsize=(5.5, 5.5))
plt.scatter(series[K_TRAIN:], pred_te, s=18)
lims = [series.min() - 0.2, series.max() + 0.2]
plt.plot(lims, lims, "r--", label="y = x")
plt.xlabel("эталон"); plt.ylabel("прогноз"); plt.title("Тест: эталон vs прогноз")
plt.legend(); plt.grid(alpha=0.3); plt.show()

fig, ax = plt.subplots(1, 2, figsize=(13, 4))
steps_axis = np.arange(1, STEPS + 1)
ax[0].plot(steps_axis, rec_true, "k", label="эталон")
ax[0].plot(steps_axis, rec_pred, "r--", label="рекурсивный прогноз")
ax[0].set_xlabel("шаг"); ax[0].set_ylabel("x"); ax[0].legend(); ax[0].grid(alpha=0.3)
ax[1].plot(steps_axis, rec_err, "o-")
ax[1].axhline(threshold, color="r", linestyle="--", label="5% размаха")
ax[1].set_xlabel("шаг"); ax[1].set_ylabel("|ошибка|"); ax[1].legend(); ax[1].grid(alpha=0.3)
plt.show()

def run_mode(w, p, series, x_min, x_max, k=None, window=None, title=""):
    if window is None:
        window = series[k - p:k]
    window = np.asarray(window, dtype=float)
    assert len(window) == p, f"нужно ровно {p} значений"
    xn = normalize(window, x_min, x_max)
    _, yn = forward(w, xn)
    y = denormalize(yn, x_min, x_max)
    print(f"--- {title}")
    print("окно (исходная шкала):", np.round(window, 4))
    print("нормализованные входы:", np.round(xn, 4))
    print(f"прогноз (норм.):   {float(yn):.4f}")
    print(f"прогноз (исходн.): {float(y):.4f}")
    if k is not None:
        print(f"истинное x({k}) = {series[k]:.4f}, абс. ошибка = {abs(series[k] - y):.4f}")

for k in [20, 70, 120]:
    run_mode(w_base, P, series, x_min, x_max, k=k, title=f"train, k={k}")
for k in [145, 170, 195]:
    run_mode(w_base, P, series, x_min, x_max, k=k, title=f"test, k={k}")

outlier = series[94:100].copy()
outlier[3] = 8.0
run_mode(w_base, P, series, x_min, x_max, k=100, window=outlier, title="окно k=100 с выбросом
8.0")
run_mode(w_base, P, series, x_min, x_max, window=[1.0, 1.5, 2.0, 1.2, 0.3, -0.5],
title="пользовательское окно")
```

Результат тестирования:

```
x_min = -2.6592, x_max = 2.6913
train в [0.000; 1.000]
test в [0.013; 0.994]
```

```
train: (134, 6) (134,) -> примеров: 134
test: (60, 6) (60,) -> примеров: 60


lr    | эпохи (ср ± σ) | test MAE | не сошлись
 0.01 |  360.0 ±  14.2 | 0.0454   | 0
 0.03 |  121.6 ±   5.0 | 0.0446   | 0
 0.05 |   74.2 ±   3.0 | 0.0433   | 0
 0.10 |   38.8 ±   1.2 | 0.0426   | 0
 0.20 |   23.2 ±   1.5 | 0.0415   | 0
 0.30 |   20.4 ±   2.1 | 0.0369   | 0


Эпох: 40, Es = 9.724e-05, критерий достигнут: True
train: MAE=0.0440  RMSE=0.0528  sMAPE=10.59%  (MAPE=37.1% — раздувается из-за
нулей)
test : MAE=0.0428  RMSE=0.0518  sMAPE=15.37%  (MAPE=48.5% — раздувается из-за
нулей)

Исследование окна (N = 5)
   p | эпохи: ср. ± σ    | test MAE: ср. ± σ| не достигли Es ≤ Ee (из 5)
   2 |  20000.0 ± 0.0    | 0.3172 ± 0.0241  | 5
   4 |    353.0 ± 14.3   | 0.0418 ± 0.0013  | 0
   6 |     38.8 ± 1.2    | 0.0426 ± 0.0023  | 0
  12 |     11.6 ± 1.6    | 0.0359 ± 0.0024  | 0

Исследование скрытого слоя (p = 6)
   N | эпохи: ср. ± σ    | test MAE: ср. ± σ| не достигли Es ≤ Ee (из 5)
   1 |     46.6 ± 2.9    | 0.0406 ± 0.0041  | 0
   2 |     44.4 ± 2.4    | 0.0401 ± 0.0025  | 0
   5 |     38.8 ± 1.2    | 0.0426 ± 0.0023  | 0
  10 |     38.2 ± 2.0    | 0.0355 ± 0.0050  | 0

Лучшая конфигурация по среднему test MAE: p = 6, N = 10
Берём её лучший запуск: seed = 3, test MAE = 0.0277

Порог 5% размаха: 0.2675
Ошибка впервые превышает порог на шаге 14 (k = 153)

--- train, k=20
окно (исходная шкала): [2.6913  2.1927  0.8359 -0.5006 -1.1143 -1.0192]
нормализованные входы: [1.     0.9068 0.6532 0.4034 0.2887 0.3065]
прогноз (норм.): 0.3357
прогноз (исходн.): -0.8630
истинное x(20) = -0.8184, абс. ошибка = 0.0446
--- train, k=70
окно (исходная шкала): [-0.729  -0.265  -0.2842 -0.2297  0.4252  1.5775]
нормализованные входы: [0.3608 0.4475 0.4439 0.4541 0.5765 0.7918]
прогноз (норм.): 0.9436
прогноз (исходн.): 2.3896
истинное x(70) = 2.4901, абс. ошибка = 0.1005
--- train, k=120
окно (исходная шкала): [1.6262  1.006  -0.4274 -1.9291 -2.5996 -2.1159]
нормализованные входы: [0.8009 0.685  0.4171 0.1365 0.0111 0.1015]
прогноз (норм.): 0.3027
прогноз (исходн.): -1.0398
истинное x(120) = -0.9702, абс. ошибка = 0.0696
--- test, k=145
окно (исходная шкала): [ 2.3554  2.6568  1.7938  0.3381 -0.8038 -1.1385]
нормализованные входы: [0.9372 0.9936 0.8323 0.5602 0.3468 0.2842]
прогноз (норм.): 0.3102
```

```
прогноз (исходн.): -0.9995
истинное x(145) = -0.9282, абс. ошибка = 0.0713
--- test, k=170
окно (исходная шкала): [0.4861 1.6362 1.8137 1.3168 0.866  0.8892]
нормализованные входы: [0.5878 0.8028 0.836  0.7431 0.6589 0.6632]
прогноз (норм.): 0.6895
прогноз (исходн.): 1.0300
истинное x(170) = 1.1256, абс. ошибка = 0.0956
--- test, k=195
окно (исходная шкала): [-1.3188 -0.505  -0.2375 -0.3083 -0.0876  0.7848]
нормализованные входы: [0.2505 0.4026 0.4526 0.4394 0.4806 0.6437]
прогноз (норм.): 0.8433
прогноз (исходн.): 1.8528
истинное x(195) = 1.9562, абс. ошибка = 0.1035
--- окно k=100 с выбросом 8.0
окно (исходная шкала): [-0.421   1.2127  2.162   8.      1.2715  0.6184]
нормализованные входы: [0.4183 0.7236 0.9011 1.9922 0.7346 0.6126]
прогноз (норм.): -0.1426
прогноз (исходн.): -3.4224
истинное x(100) = 0.4899, абс. ошибка = 3.9122
--- пользовательское окно
окно (исходная шкала): [ 1.   1.5  2.   1.2  0.3 -0.5]
нормализованные входы: [0.6839 0.7773 0.8708 0.7213 0.5531 0.4036]
прогноз (норм.):0.3257
прогноз (исходн.): -0.9163
```

График с отметкой границы между обучающим и тестовым участками:

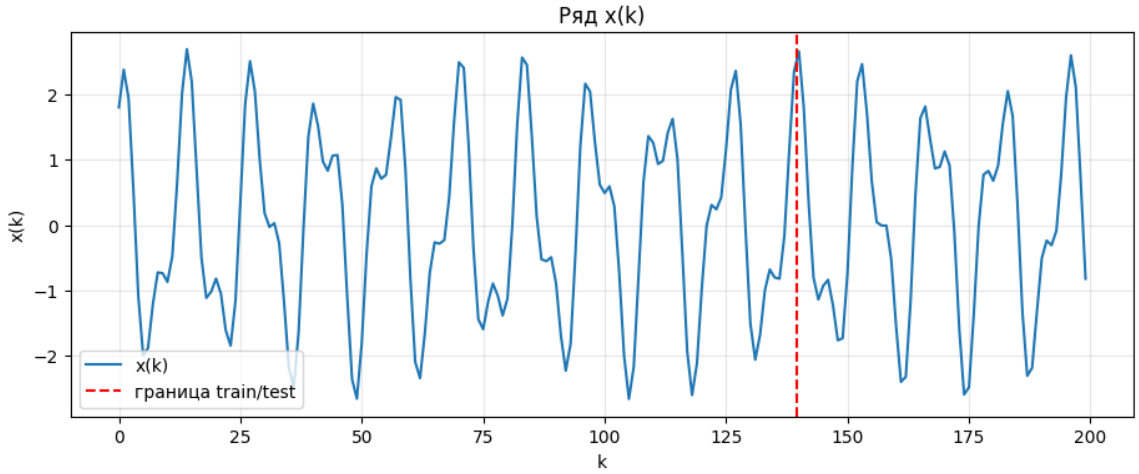
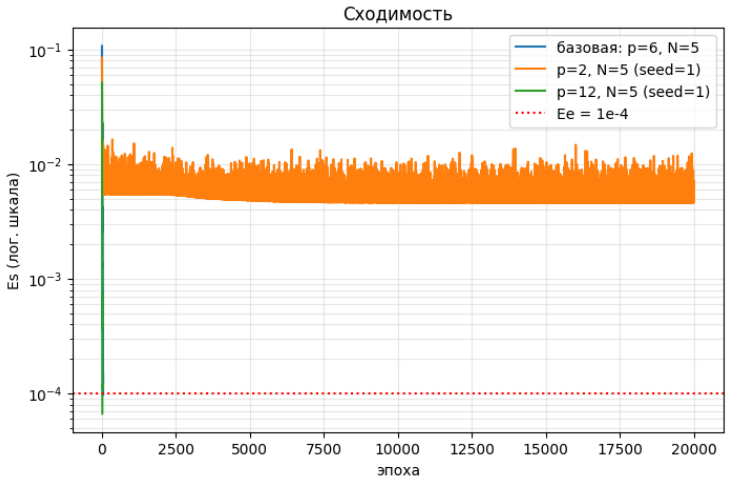


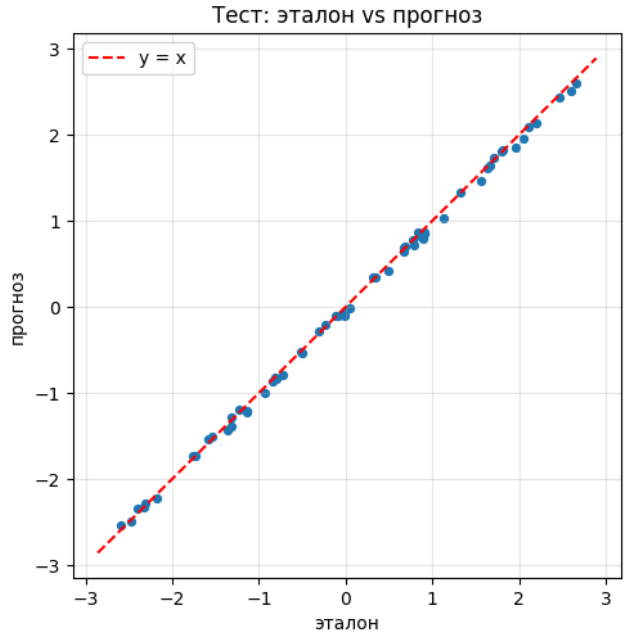
График с одношаговым прогнозом:



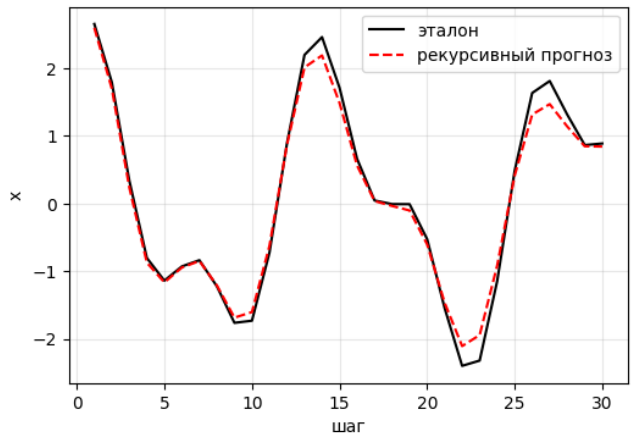
Сходимость:

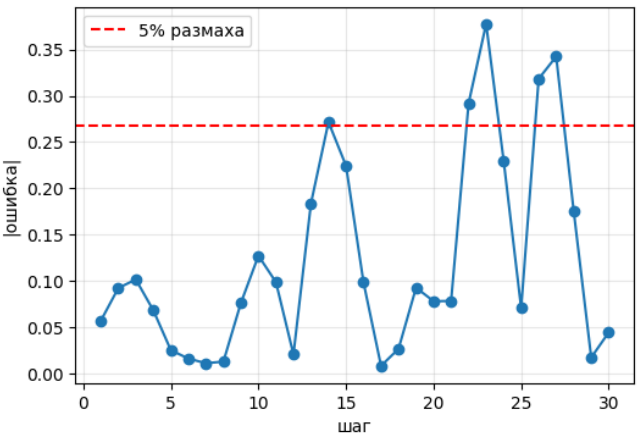


Тест «Эталон – прогноз»:



Рекурсивный прогноз и рост ошибки по шагам:





Ручной расчет одного шага обучения сети 3-2-1 при V = 8:

Дано: x = [0.2; 0.5; 0.8], d = 0.6
Скрытый слой: W11 = 0.4, W12 = -0.2, W21 = 0.3, W22 = 0.4, W31 = -0.1, W32=0.4, b1=0.1, b2=-0.1
Выходной слой: W_out1 = 0.5, W_out2 = -1.6, b_out = 0.2

Forward
s1 = 0.2·0.4 + 0.5·0.3 + 0.8 · (-0.1) + 0.1 = 0.08 + 0.15 - 0.08 + 0.1 = 0.25
s2 = 0.2 · (-0.2) + 0.5·0.4 + 0.8·0.4 + (-0.1) = -0.04 + 0.2 + 0.32 - 0.1 = 0.38

h1 = 1 / (1+$e^{-0.25}$) = 1 / 1.7788 = 0.5622
h2 = 1 / (1+$e^{-0.38}$) = 1 / 1.6839 = 0.5939

y = 0.5 · 0.5622 + (-1.6) · 0.5939 + 0.2
= 0.2811 - 0.9502 + 0.2 = -0.4691 (линейный выход)

E = (0.6 - (-0.4691))^2 = 1.0691^2 = 1.1430

Backward
δ_out = 2(y - d) = 2(-0.4691 - 0.6) = -2.1382

Выходной слой:
dW_out1 = δ_out· h1 = -2.1382 · 0.5622 = -1.2021
dW_out2 = δ_out· h2 = -2.1382 · 0.5939 = -1.2698
db_out = δ_out = -2.1382

Скрытый слой (берём старые W_out):
δ1 = δ_out · W_out1· h1(1-h1)
= -2.1382 · 0.5 · 0.5622 · 0.4378 = -0.2631
δ2 = δ_out · W_out2· h2(1-h2)
= -2.1382 · (-1.6) · 0.5939 · 0.4061 = 0.8251

dW11 = 0.2 · δ1 = -0.0526 dW12 = 0.2 · δ2 = 0.1650
dW21 = 0.5 · δ1 = -0.1316 dW22 = 0.5 · δ2 = 0.4126
dW31 = 0.8 · δ1 = -0.2105 dW32 = 0.8 · δ2 = 0.6601
db1 = δ1 = -0.2631 db2 = δ2 = 0.8251

Обновление
w_new = w – η · dw, η = 0.1
W_out1 = 0.5 + 0.12021 = 0.6202
W_out2 = -1.6 + 0.12698 = -1.4730
B_out = 0.2 + 0.21382 = 0.4138
W11 = 0.4 + 0.00526 = 0.4053
W21 = 0.3 + 0.01316 = 0.3132
W31 = -0.1 + 0.02105 = -0.0789
W12 = -0.2 - 0.01650 = -0.2165
W22 = 0.4 - 0.04126 = 0.3587
W32 = 0.4 - 0.06601 = 0.3340
b1 = 0.1 + 0.02631 = 0.1263
b2 = -0.1 - 0.08251 = -0.1825

Проверка: прогон с новыми весами даёт y = -0.0473,
E = 0.4189 < 1.143
То есть ошибка уменьшилась.

Вывод: Проделав данную лабораторную работу я научился формировать обучающую выборку методом скользящего окна, нормализовать данные, оценил качество одношагового и многошагового прогноза и исследовал влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.